

VAYU-X

Autonomous Surveillance UAV + AI Visual Intelligence Ground Station

VAYU-X is a hybrid VTOL fixed-wing autonomous surveillance platform designed for aerial monitoring, terrain reconnaissance, disaster-response support, mission intelligence, and AI-assisted evidence analysis.

The system combines an airborne sensing platform with a ground-based AI command center. The aircraft provides live video, GPS/flight telemetry, and mission data, while the ground station performs real-time object detection, tracking, incident generation, image enhancement, OCR, Visual Intelligence, and optional public-web intelligence.

1. Project Overview

VAYU-X is designed as a modular autonomous aerial surveillance system with two major layers:

1. Airborne Platform

- Hybrid VTOL fixed-wing aircraft
- Pixhawk 2.4.8 flight controller
- Raspberry Pi 5 companion computer
- GPS/GNSS
- SIYI A8 Mini 6× zoom onboard camera
- 3DR 433 MHz telemetry
- HT-10A transmitter/receiver
- Brushless propulsion
- Servos and control surfaces
- Independent video communication channel

2. AI Ground Station

- FastAPI backend
- React + TypeScript dashboard
- OpenCV
- YOLOv8
- DeepSort
- SQLite
- Incident management
- Image evidence storage
- Visual Intelligence
- EasyOCR
- Image enhancement
- Region-of-interest analysis
- QR decoding

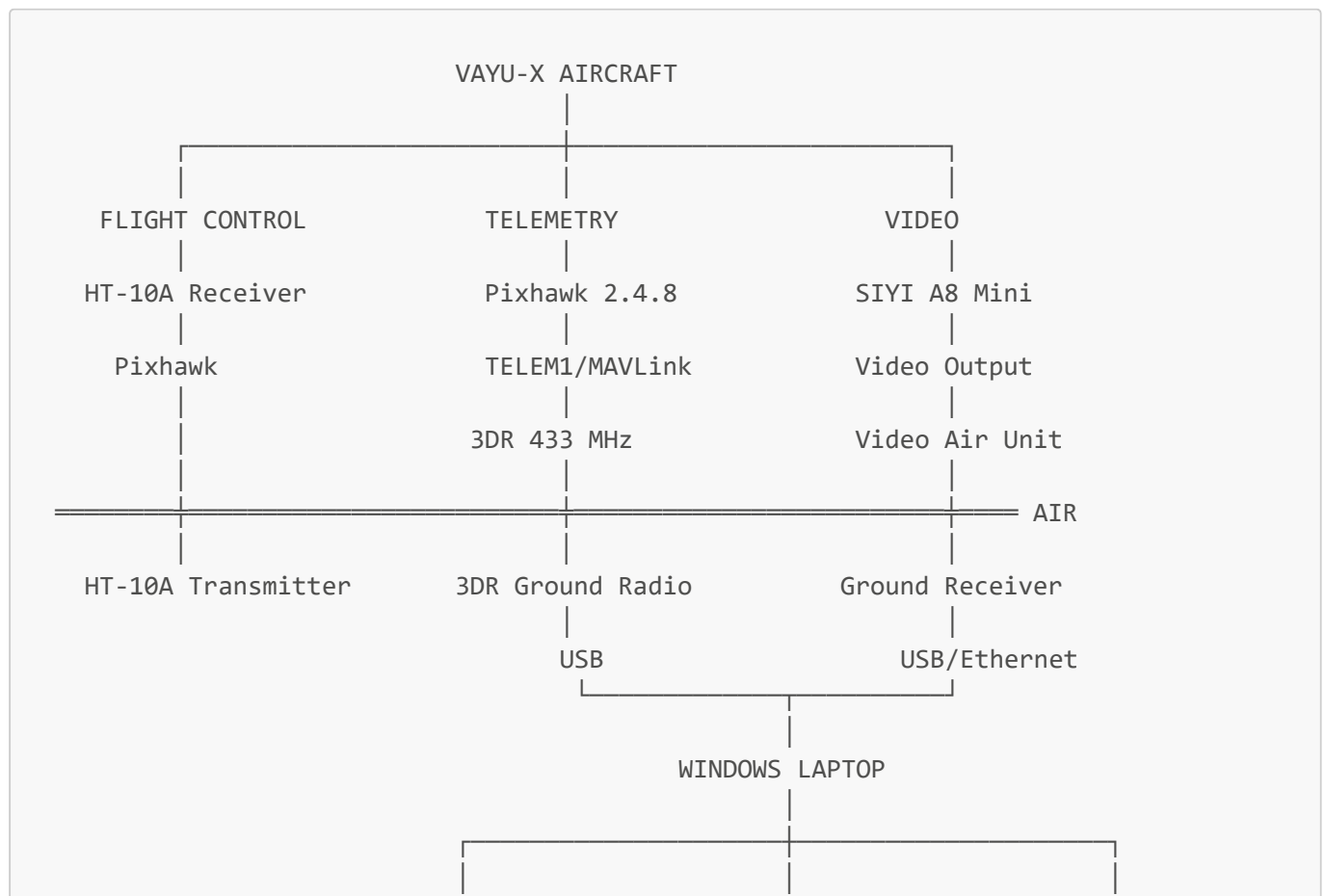
- Web Intelligence provider architecture
- Brave Search integration when configured

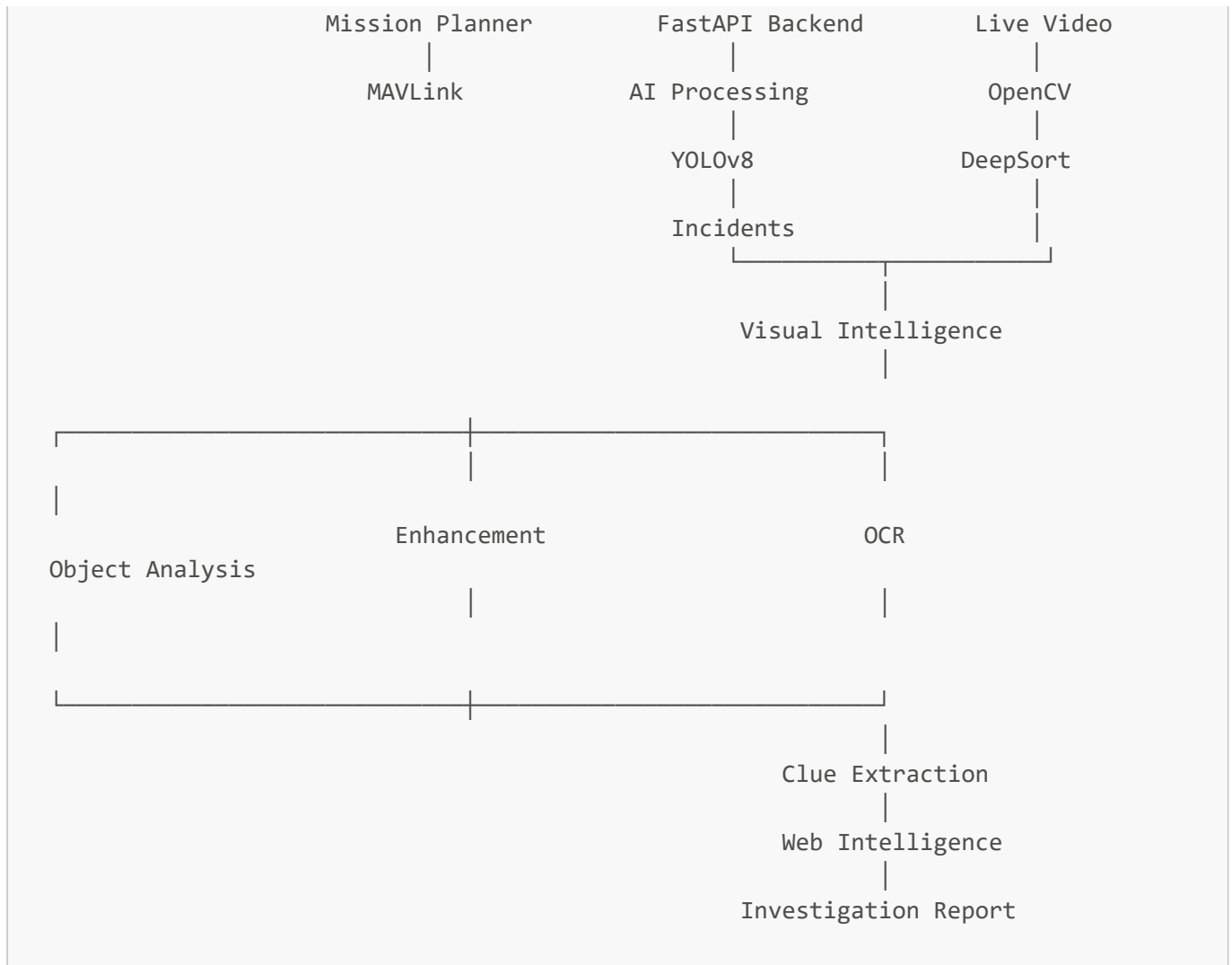
2. Main Objective

The project aims to provide a practical, modular and affordable airborne intelligence platform capable of:

- autonomous or pilot-assisted flight;
- live aerial video monitoring;
- GPS-based mission awareness;
- object detection and tracking;
- automatic evidence capture;
- surveillance incident generation;
- image enhancement;
- OCR and text extraction;
- region-based image investigation;
- object and clue extraction;
- web-assisted investigation from extracted text;
- mission replay and evidence review;
- operation without onboard internet or Wi-Fi.

3. High-Level System Architecture





4. Why the System Does Not Require Wi-Fi on the Aircraft

The aircraft does **not** need normal Wi-Fi or an internet connection.

VAYU-X separates communication into independent channels:

Link	Purpose	Example Hardware
RC Control	Manual control / pilot input	HT-10A transmitter + receiver
Telemetry	GPS, altitude, speed, battery, mode	3DR 433 MHz telemetry
Video	Live surveillance feed	Dedicated video air/ground link
Internet	Optional Web Intelligence	Ground laptop only

The important design principle is:

Flight control, telemetry, video, AI, and internet should not depend on one single communication path.

This allows the aircraft to remain controllable even if the video or AI subsystem fails.

5. Hardware Architecture

5.1 Flight Controller

Pixhawk 2.4.8

Main responsibilities:

- aircraft stabilization;
- navigation;
- GPS integration;
- motor/ESC control;
- servo control;
- autonomous waypoint missions;
- return-to-launch;
- geofence;
- flight-mode control;
- telemetry exchange.

Recommended logical connections:

```
Pixhawk
|
├── GPS Port      → GPS/GNSS module
├── TELEM1        → 3DR 433 MHz Air Radio
├── TELEM2        → Raspberry Pi 5
├── RC Input      → HT-10A Receiver
├── MAIN/AUX OUT  → ESCs / Servos
└── Power         → regulated avionics supply
```

5.2 Companion Computer

Raspberry Pi 5

Recommended role:

- receive MAVLink telemetry from Pixhawk;
- camera/gimbal interfacing;
- local metadata collection;
- local health monitoring;
- optional edge processing;
- onboard logging;
- future lightweight AI inference;
- communication gateway;

- forwarding mission metadata.

The Raspberry Pi should **not** initially become the safety-critical flight controller.

Pixhawk remains responsible for safe flight.

5.3 Camera

SIYI A8 Mini 6× Zoom

Primary role:

- aerial surveillance;
- zoom inspection;
- visual evidence capture;
- AI input;
- OCR input;
- Visual Intelligence analysis.

Example workflow:

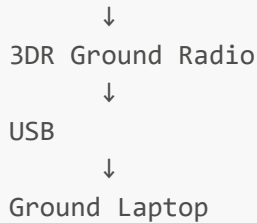
```
Wide-area observation
  ↓
Object detected
  ↓
Operator selects target
  ↓
Camera zoom
  ↓
Capture frame
  ↓
Image enhancement
  ↓
OCR / object analysis
  ↓
Visual Intelligence
```

5.4 Telemetry

3DR 433 MHz Telemetry

Recommended use:

```
Pixhawk TELEM1
  ↓
3DR Air Radio
  ↓
433 MHz RF
```



Telemetry can carry:

- latitude;
- longitude;
- altitude;
- heading;
- ground speed;
- battery information;
- GPS fix;
- flight mode;
- armed status;
- mission state;
- system health.

Do **not** use this link for normal live video.

5.5 RC Control

HT-10A Transmitter + F-10A Receiver

Recommended role:

- pilot/manual control;
- emergency pilot intervention;
- independent flight-control path.

Do not make primary RC control dependent on:

- Raspberry Pi;
 - FastAPI;
 - YOLO;
 - laptop internet;
 - Web Intelligence.
-

6. Recommended Video Communication Architecture

The hardware proposal already separates video from telemetry.

Recommended logical path:

```
SIYI A8 Mini
  ↓
Video Output
  ↓
Dedicated Video Air Unit
  ↓
Wireless Video Link
  ↓
Ground Video Unit
  ↓
Laptop
  ↓
OpenCV / FastAPI
```

Possible implementation paths:

Option A — IP / Ethernet Video

```
Camera
  ↓
IP video
  ↓
Digital Air Link
  ↓
Digital Ground Link
  ↓
Ethernet
  ↓
Laptop
  ↓
RTSP/OpenCV
```

Recommended when the chosen camera/link supports network video.

Option B — HDMI Video Link

```
Camera HDMI
  ↓
HDMI transmitter
  ↓
Wireless link
  ↓
HDMI receiver
  ↓
USB capture card
  ↓
Windows
```

↓
OpenCV

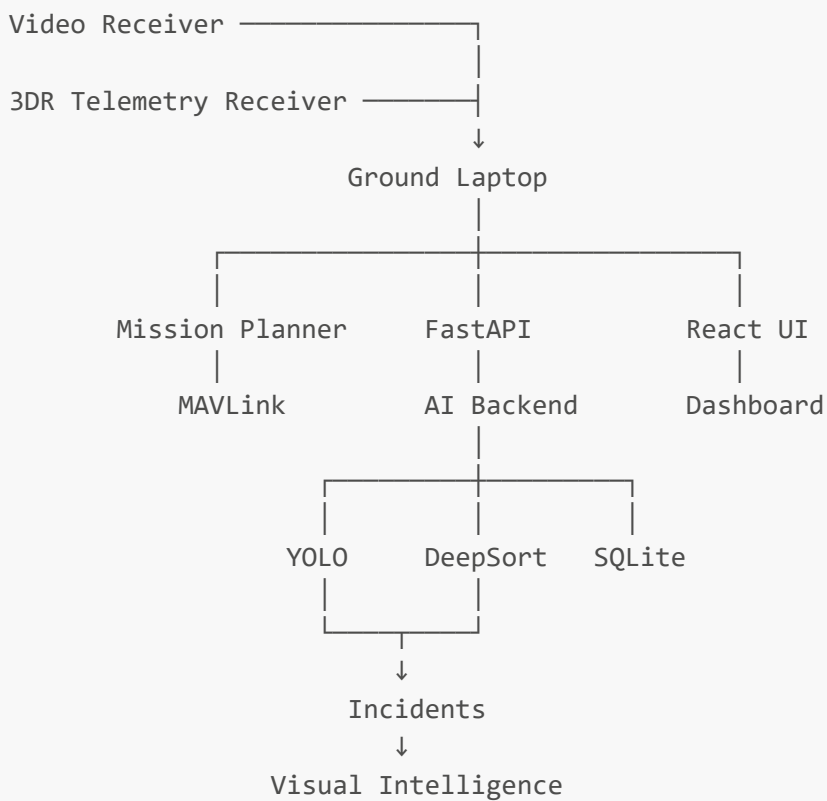
Option C — Analog/FPV + Capture

Useful only if required by the selected video hardware.

Camera
↓
Video transmitter
↓
FPV receiver
↓
USB capture card
↓
OpenCV

7. Ground Station Architecture

The ground laptop performs the heavy AI and application work.



8. Mission Planner + VAYU-X AI Dashboard

Mission Planner and the custom VAYU-X dashboard should be used together.

Mission Planner

Recommended for:

- Pixhawk setup;
- waypoint creation;
- geofence;
- flight modes;
- mission upload;
- autopilot parameters;
- arming/disarming;
- RTL configuration;
- flight logs.

VAYU-X AI Command Center

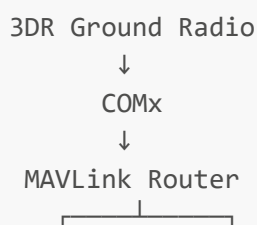
Recommended for:

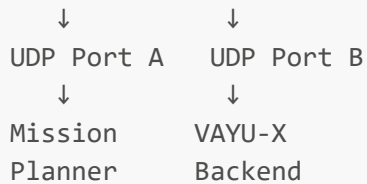
- live video;
- object detection;
- person/object tracking;
- AI alerts;
- incident management;
- GPS-tagged evidence;
- OCR;
- image enhancement;
- Visual Intelligence;
- Web Intelligence;
- mission investigation.

9. MAVLink Fan-Out

Do not allow Mission Planner and the VAYU-X backend to fight for the same COM port.

Use a MAVLink router/forwarder:

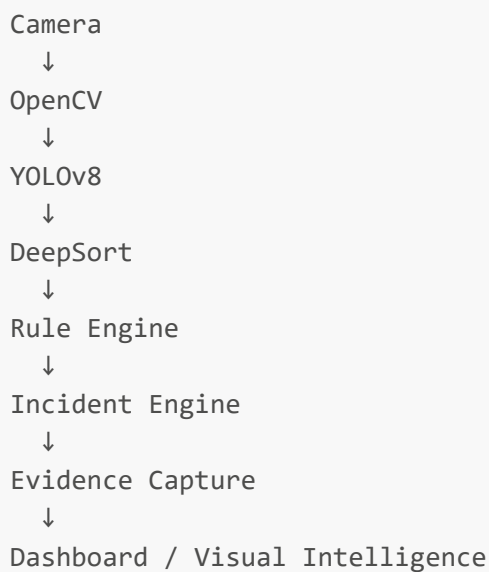




This allows both applications to receive the same telemetry stream.

10. AI Surveillance Pipeline

Current logical pipeline:



Typical detections may include:

- person;
- car;
- truck;
- bus;
- motorcycle;
- bicycle;
- other YOLO-supported classes.

The system should avoid generating an incident on every frame. Use:

- cooldowns;
- rule conditions;
- zones;
- confidence thresholds;
- manual capture;
- event persistence.

11. Visual Intelligence V1

The project includes a Visual Intelligence subsystem.

Main features:

- secure image upload;
- immutable original evidence;
- SHA-256 hashing;
- image enhancement;
- OCR;
- OCR bounding boxes;
- object detection;
- QR decoding;
- clue extraction;
- region-of-interest analysis;
- incident-image investigation;
- derivative image generation;
- report generation.

12. Image Enhancement

Supported operations include:

- auto contrast;
- CLAHE;
- brightness;
- contrast;
- gamma correction;
- sharpening;
- denoising;
- grayscale;
- deskew;
- resizing/upscale;
- perspective correction where appropriate.

Important evidence rule:

The original evidence image must never be overwritten.

Use separate labels such as:

ORIGINAL EVIDENCE
ENHANCED DERIVATIVE
OCR OVERLAY

```
OBJECT OVERLAY
ROI CROP
```

13. OCR

EasyOCR

Use cases:

- signboards;
- product labels;
- equipment markings;
- building names;
- model numbers;
- website/domain text;
- emails;
- documents;
- banners;
- registration-like strings;
- QR-associated text.

OCR runs locally after models are cached.

Suggested configuration:

```
OCR_ENABLED=true
OCR_MODEL_DOWNLOAD_ENABLED=false
OCR_MIN_CONFIDENCE=0.35
```

For first-time model download:

```
OCR_MODEL_DOWNLOAD_ENABLED=true
```

Start backend, initialize OCR once, then return to:

```
OCR_MODEL_DOWNLOAD_ENABLED=false
```

14. Region-of-Interest Analysis

Analysts can select only a useful portion of a frame.

Example:

```
Full aerial frame
  ↓
Select building sign
  ↓
Crop region
  ↓
Enhance
  ↓
OCR
  ↓
Extract clue
  ↓
Investigation
```

This is useful when text is too small in the complete image.

15. Web Intelligence V1.5

Web Intelligence is optional and runs from the **ground station**.

Pipeline:

```
Image
  ↓
Local OCR / Object Detection
  ↓
Textual clues
  ↓
Search query generation
  ↓
Search provider
  ↓
Source collection
  ↓
Evidence correlation
  ↓
Investigation report
```

The system does not need to upload the original image to the web-search provider.

16. Brave Search Configuration

Example:

```
WEB_SEARCH_ENABLED=true
WEB_SEARCH_PROVIDER=brave
WEB_SEARCH_API_KEY=<secret>
WEB_SEARCH_MAX_QUERIES=3
WEB_SEARCH_RESULTS_PER_QUERY=5
WEB_SEARCH_TIMEOUT_SECONDS=8
```

Never commit the real key.

When disabled:

```
WEB_SEARCH_ENABLED=false
WEB_SEARCH_PROVIDER=disabled
```

The rest of Visual Intelligence must continue to work.

17. Evidence Separation

The investigation engine should separate three categories:

Observed

Directly visible or detected:

- OCR text;
- YOLO objects;
- QR data;
- timestamps;
- image metadata.

Retrieved

Information returned from external search:

- title;
- URL;
- domain;
- snippet;
- provider.

Inferred

Candidate conclusions generated by correlating evidence.

Example:

Observed:

- OCR: LENOVO
- OCR: T14 GEN 5
- Object: laptop

Retrieved:

- official Lenovo page
- technical documentation

Inferred:

- likely Lenovo ThinkPad T14 Gen 5

Confidence:

HIGH

The system should not present uncertain matches as guaranteed facts.

18. Privacy Boundary

Do not implement:

- unknown-person facial identification;
- reverse-face lookup;
- biometric profiling;
- internet identity lookup from faces.

Do not automatically send:

- aircraft GPS;
- original images;
- enhanced images;
- cropped images;
- telemetry;
- user credentials;
- JWTs

to Web Intelligence providers.

External search should receive only analyst-approved textual queries.

19. Airborne Evidence Model

Recommended evidence metadata:

```
{  
  "evidence_id": "AIR-EVIDENCE-0042",  
  "camera_id": "AIR-01",  
  "mission_id": "MISSION-001",  
  "capture_timestamp": "2026-09-04T10:42:17.200",  
  "latitude": 22.000000,  
  "longitude": 73.000000,  
  "altitude_m": 84.2,  
  "heading_deg": 142.0,  
  "sha256": "...",  
  "image_path": "..."  
}
```

If nearby telemetry is unavailable, the system must show:

```
GPS: Unavailable
```

Do not invent coordinates.

20. Timestamp Correlation

Video frames and MAVLink telemetry will not arrive at exactly the same moment.

Use nearest-sample matching:

```
Frame Time:  
10:42:17.200  
  
Nearest Telemetry:  
10:42:17.430  
  
Difference:  
230 ms
```

Recommended fields:

- video timestamp;
- telemetry timestamp;
- time difference;
- GPS;
- altitude;
- heading;
- speed.

21. Communication Health Monitoring

Recommended dashboard:

AIR-01

RC Link	Healthy
Telemetry	Healthy
Video	Healthy
GPS	3D Fix
AI	Running

Telemetry Age	0.3 s
Frame Age	0.06 s
FPS	15.4
Reconnect Count	0

Recommended states:

ONLINE
DEGRADED
RECONNECTING
OFFLINE

Each link should have its own health state.

Video failure must not automatically mean RC/telemetry failure.

22. Communication Failure Events

Example:

COMMUNICATION EVENT

AIR-01
Video Link Lost

Time:
10:42:19

Telemetry:
ONLINE

Last known altitude:
84.0 m

```
Last frame:
stored as evidence
```

Recovery:

```
AIR-01
Video Link Restored

Downtime:
8.2 seconds
```

The same model may be used for telemetry loss.

23. Degraded-Link Strategy

Future design:

```
STRONG LINK
→ HD video + telemetry + metadata

MODERATE LINK
→ reduced bitrate video + telemetry

POOR LINK
→ low-rate video / snapshots + telemetry

VERY POOR LINK
→ telemetry + AI metadata

VIDEO LOST
→ telemetry only
```

Do not implement adaptive switching until the base communication system is stable.

24. Onboard Recording

Never depend only on the wireless video stream.

Recommended:

```
Camera
|
```

- └ Live feed → Ground Station
- └ Local recording → onboard storage

After landing:

```
Recorded mission video
  ↓
Upload to Ground Station
  ↓
Recorded Mission Analysis
  ↓
YOLO
  ↓
Interesting frames
  ↓
Visual Intelligence
```

25. Optional Separate Pilot Camera

A useful advanced configuration:

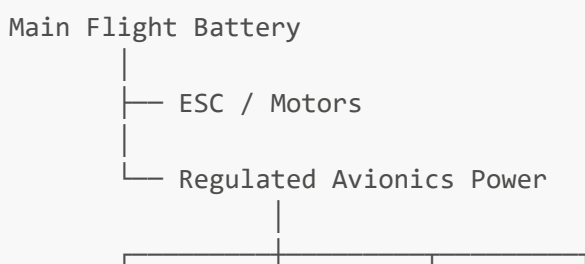
```
Camera 1:
Wide-angle FPV
→ Pilot orientation

Camera 2:
SIYI A8 Mini
→ Surveillance / Zoom / AI
```

This prevents a zoomed or off-axis surveillance camera from becoming the pilot's only visual reference.

26. Power Architecture

Recommended concept:



↓ ↓ ↓ ↓
Pixhawk Raspberry Pi Camera Video Link

Avoid powering sensitive electronics from an unstable supply.

Consider:

- BEC sizing;
- current headroom;
- filtering;
- grounding;
- connector quality;
- motor/ESC noise.

27. Antenna Placement

Keep RF antennas separated from:

- ESCs;
- motor wiring;
- high-current power wiring;
- switching regulators;
- carbon-fiber structures where possible.

Plan separately for:

- GPS antenna;
- RC antenna;
- 433 MHz telemetry antenna;
- video-link antenna.

Follow hardware manufacturer guidance for antenna orientation and RF installation.

28. Flight Safety

The aircraft must remain safe even if the AI stack fails.

```
Pixhawk
|
|— stabilization
|— navigation
|— geofence
|— RC failsafe
|— battery failsafe
```

```
├─ RTL
├─ landing logic
```

Independent AI subsystem:

```
Raspberry Pi / Ground AI
├─ YOLO
├─ OCR
├─ Visual Intelligence
├─ Web Intelligence
├─ Dashboard
```

Failures in the AI application must not prevent safe Pixhawk behavior.

29. Software Stack

Backend

- Python
- FastAPI
- OpenCV
- Ultralytics YOLOv8
- DeepSort
- EasyOCR
- Pillow
- SQLite
- python-multipart

Frontend

- React
- TypeScript
- Vite

Flight / Telemetry

- Pixhawk
 - MAVLink
 - Mission Planner / compatible ground-control tooling
-

30. Backend Setup — Windows

From the project root:

```
cd C:\Smartsurveillance_System_OpenCV-main
```

Activate backend virtual environment:

```
.\backend\.venv\Scripts\Activate.ps1
```

Start backend:

```
.\scripts\start_backend.ps1
```

Expected backend:

```
http://127.0.0.1:8001
```

31. Frontend Setup

Start frontend:

```
.\scripts\start_frontend.ps1
```

Open:

```
http://localhost:5173
```

32. Core Environment Configuration

Example local/trusted-LAN configuration:

```
CAMERA_SOURCE=0  
  
YOLO_MODEL=backend/yolov8n.pt  
CONFIDENCE_THRESHOLD=0.5
```

```
DATABASE_PATH=backend/surveillance.db
CAPTURE_DIRECTORY=outputs

TELEGRAM_ENABLED=false

CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,http://127.0.0.1:8001

LOG_LEVEL=INFO
CAMERA_RETRY_SECONDS=3

VITE_API_BASE_URL=http://127.0.0.1:8001

AUTH_ENABLED=true
PROTECT_MONITORING_ROUTES=false

COOKIE_SECURE=false
```

Do not place passwords, secrets or real API keys in `.env.example`.

33. Camera Source Examples

Laptop webcam:

```
CAMERA_SOURCE=0
```

USB capture card:

```
CAMERA_SOURCE=1
```

Network/RTSP camera:

```
CAMERA_SOURCE=rtsp://<camera-address>/<stream>
```

For airborne operation, use the ground-received video source visible to Windows/OpenCV.

34. Authentication Model

Current trusted-LAN behavior:

```
AUTH_ENABLED=true  
PROTECT_MONITORING_ROUTES=false
```

Read-only monitoring can remain available to the dashboard, while mutation/admin routes remain protected.

For internet-facing deployment:

```
AUTH_ENABLED=true  
PROTECT_MONITORING_ROUTES=true  
COOKIE_SECURE=true  
HTTPS enabled
```

Use secure session/stream authentication.

35. Visual Intelligence Storage

Recommended structure:

```
outputs/  
└─ visual_intelligence/  
    ├── originals/  
    ├── enhanced/  
    ├── overlays/  
    ├── crops/  
    └─ models/  
        └─ easyocr/
```

Original evidence is immutable.

36. Testing

Current project testing should include:

- auth;
- route policy;
- camera behavior;
- incident persistence;
- zone/rule security;
- Visual Intelligence upload validation;
- SHA-256;
- immutable originals;

- ROI validation;
- OCR mocked behavior;
- object detection;
- entity extraction;
- web-provider behavior;
- source deduplication;
- protected mutation endpoints;
- surveillance regression.

Backend:

```
.\backend\.venv\Scripts\python.exe -m pytest backend\tests -q
```

Frontend:

```
cd frontend  
npm run lint  
npm run build
```

37. Current AI/Visual Intelligence Verification State

The project has been validated through:

- backend automated tests;
- real FastAPI image-upload smoke testing;
- YOLO still-image analysis on CPU;
- enhancement derivative validation;
- immutable evidence verification;
- ROI analysis;
- incident-to-Visual-Intelligence linkage;
- frontend lint/build;
- surveillance endpoint regression testing.

38. Recommended Airborne Software Phase

Future module:

```
AIRBORNE INTELLIGENCE  
|  
├─ aircraft source model  
├─ MAVLink telemetry receiver
```

- |— telemetry persistence
- |— mission sessions
- |— GPS-tagged incidents
- |— mission map
- |— flight track
- |— communication-health monitoring
- |— manual frame capture
- |— telemetry/video timestamp correlation
- |— Visual Intelligence integration

39. Suggested Airborne API Design

Future examples:

```
POST /telemetry/aircraft/{camera_id}
GET  /telemetry/aircraft/{camera_id}/latest

POST /missions
POST /missions/{id}/start
POST /missions/{id}/end

GET  /missions/{id}
GET  /missions/{id}/track

POST /airborne/{camera_id}/capture
```

40. Mission Model

Suggested mission fields:

```
mission_id
name
aircraft_camera_id
started_at
ended_at
status
```

Possible states:

```
planned
active
```

```
completed
aborted
```

41. Mission Timeline

Example:

```
10:41:00 Mission started
10:42:14 Vehicle detected
10:42:15 Evidence captured
10:42:19 Visual analysis created
10:45:03 Video link degraded
10:45:11 Video restored
10:52:31 Mission completed
```

42. Mission Track

Future backend response:

```
[
  {
    "timestamp": "2026-09-04T10:41:00",
    "latitude": 22.000001,
    "longitude": 73.000001,
    "altitude_m": 78.2
  },
  {
    "timestamp": "2026-09-04T10:41:02",
    "latitude": 22.000010,
    "longitude": 73.000020,
    "altitude_m": 80.0
  }
]
```

43. Use Cases

VAYU-X is intended for lawful and authorized applications such as:

- disaster assessment;
- search-and-rescue support;
- terrain reconnaissance;

- infrastructure inspection;
- perimeter monitoring;
- environmental observation;
- mission recording;
- authorized security monitoring;
- aerial mapping support;
- remote-area situational awareness.

44. Known Limitations

- DeepSort performance can degrade when the airborne camera is moving quickly.
- Camera-motion compensation is not yet implemented.
- OCR quality depends on image resolution, blur, angle and lighting.
- Image enhancement cannot recover information that never existed in the original frame.
- YOLO classification is probabilistic.
- Web matches are candidate correlations, not guaranteed identities.
- Video-link performance depends on the selected air/ground radio hardware.
- No unknown-person face identification is supported.
- Public-web intelligence requires internet access on the ground station.
- Ground AI should not be treated as the aircraft's safety-critical control system.

45. Recommended Development Roadmap

PHASE 1

Core Surveillance

- ✓ Camera
- ✓ YOLO
- ✓ DeepSort
- ✓ Incidents
- ✓ Zones
- ✓ Dashboard



PHASE 2

Visual Intelligence

- ✓ Image upload
- ✓ Enhancement
- ✓ OCR architecture
- ✓ ROI
- ✓ Object analysis
- ✓ Evidence storage



PHASE 3

Web Intelligence

- ✓ Search-provider architecture
- ✓ Evidence-ranked queries
- ✓ Source correlation
- ✓ Candidate assessment

↓

PHASE 4

Airborne Intelligence

- MAVLink ground integration
- GPS-tagged evidence
- missions
- mission map
- communications health

↓

PHASE 5

Physical Airborne Integration

- SIYI camera
- video air/ground link
- Pixhawk telemetry
- Raspberry Pi companion
- laptop command center

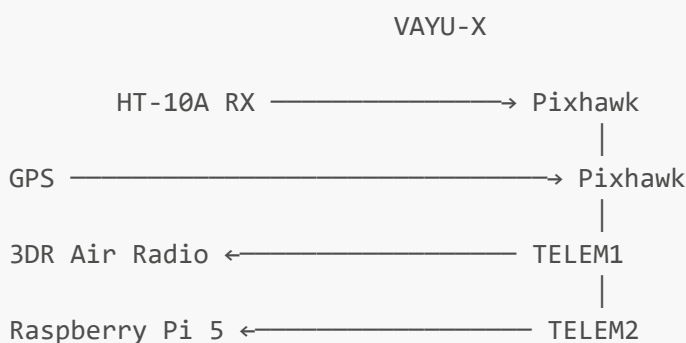
↓

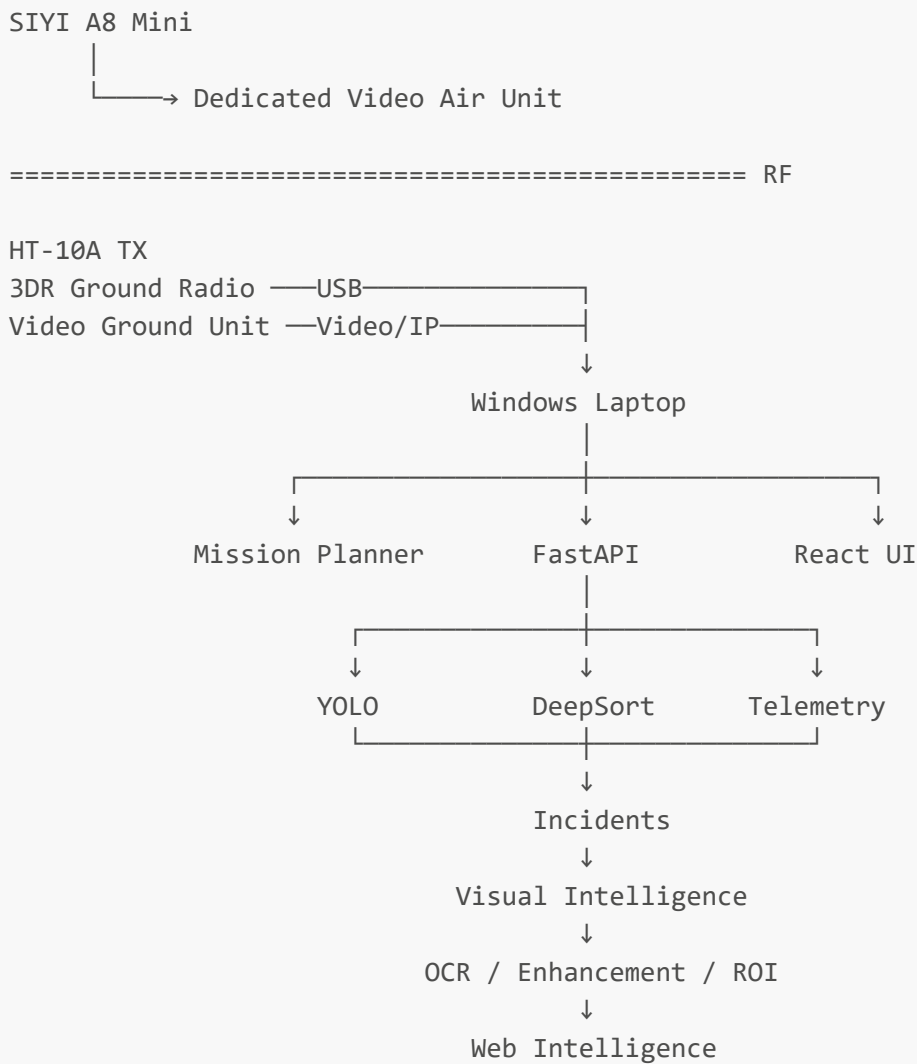
PHASE 6

Advanced Airborne AI

- camera-motion compensation
- optical flow
- onboard lightweight detection
- adaptive video
- metadata fallback
- mission replay

46. Recommended Physical Prototype Architecture





47. Communication Design Rules

1. **Keep RC independent.**
2. **Use telemetry for aircraft state, not video.**
3. **Use a dedicated video link.**
4. **Keep Pixhawk responsible for safety.**
5. **Keep Web Intelligence on the ground station.**
6. **Record important evidence locally where possible.**
7. **Log link health and reconnects.**
8. **Never fabricate GPS when telemetry is unavailable.**
9. **Do not automatically send GPS/images to external search providers.**
10. **Treat all AI conclusions as probabilistic unless independently verified.**

48. Project Identity

Project Name: VAYU-X

Category: Autonomous Surveillance UAV

Platform: Hybrid VTOL Fixed-Wing

Ground AI: Smart Surveillance + Visual Intelligence

Flight Controller: Pixhawk 2.4.8

Companion Computer: Raspberry Pi 5

Telemetry: 3DR 433 MHz

Camera: SIYI A8 Mini 6× Zoom

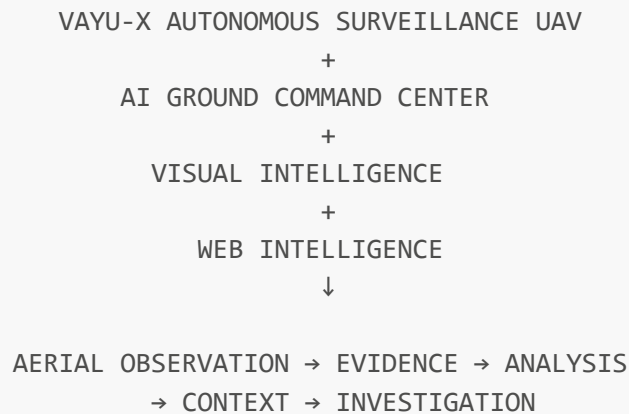
AI: YOLOv8 + OpenCV + DeepSort + EasyOCR

Backend: FastAPI

Frontend: React + TypeScript

Database: SQLite

49. Final Vision



VAYU-X is intended to demonstrate how a low-cost autonomous aerial platform can be combined with modern computer vision, mission telemetry, evidence management, OCR, and source-grounded visual investigation while keeping flight safety and communication architecture modular.

50. Disclaimer

Operate the aircraft only in accordance with applicable aviation, RF-spectrum, privacy, safety, and local operating rules.

The AI, OCR, enhancement, tracking, and web-correlation components are decision-support tools. They should not be treated as infallible identification or safety-critical flight-control systems.

Unknown-person facial identification and reverse-face lookup are intentionally outside the scope of this project.